ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORES

Laboratorio N.º 04

Temporizador regresivo y contador de eventos en FPGA

Código: CIT2109
Docente: Mg. Ing. Jorge Tobar Pinto
Software: Xilinx Vivado
Lenguaje: VHDL
Plataforma: FPGA Nexys 4 DDR – Artix-7 XC7A100T

---

1. Objetivo

Diseñar, simular e implementar en una FPGA un sistema digital compuesto por:

- un temporizador regresivo;
- un contador de eventos;
- controles de inicio, pausa y reinicio;
- visualización en cuatro displays de siete segmentos;
- una señal de alarma al finalizar el tiempo.

2. Descripción del sistema

El sistema deberá mostrar cuatro dígitos con el siguiente formato:

```
TT EE
```

donde:

- **TT**: tiempo restante, entre `00` y `99`;
- **EE**: cantidad de eventos, entre `00` y `99`.

Por ejemplo, la visualización `25 07` representa 25 segundos restantes y 7 eventos registrados.

3. Funcionamiento requerido

3.1. Temporizador

El temporizador deberá:

1. iniciar desde un valor definido;
2. disminuir una unidad cada segundo;
3. permitir inicio y pausa;
4. permitir reinicio;

5. detenerse en `00`;
6. activar una alarma al finalizar.

3.2. Contador de eventos

El contador deberá:

1. iniciar en `00`;
2. aumentar una unidad con cada pulsación;
3. contar hasta `99`;
4. permitir reinicio independiente;
5. evitar múltiples incrementos por una sola pulsación.

4. Entradas y salidas

4.1. Entradas

Señal	Descripción
`CLK`	Reloj principal de la FPGA.
`RESET`	Reinicio general.
`START_PAUSE`	Inicia o pausa el temporizador.
`RESET_TIEMPO`	Reinicia el temporizador.
`EVENTO`	Incrementa el contador de eventos.
`RESET_EVENTO`	Reinicia el contador de eventos.

4.2. Salidas

Señal	Descripción
`SEG(6 downto 0)`	Segmentos del display.
`AN(7 downto 0)`	Selección de display.
`ALARMA`	Indica tiempo agotado.
`RUNNING`	Indica que el temporizador está activo.
`LED_EVENTO`	Indica detección de evento.

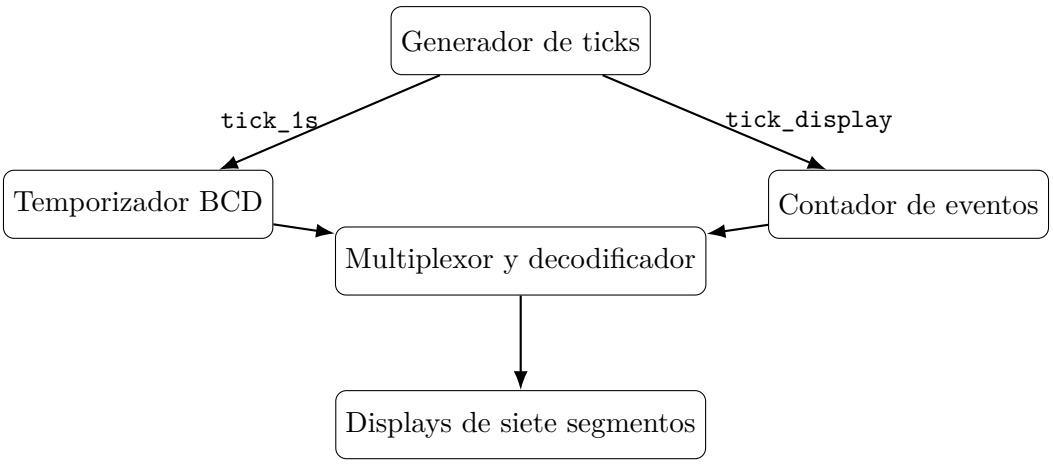
5. Asignación sugerida en la FPGA

Elemento físico	Función
`CLK100MHZ`	Reloj del sistema.
`BTNC`	Inicio y pausa.
`BTNU`	Reset del tiempo.
`BTNR`	Incremento de evento.
`BTNL`	Reset de eventos.
`BTND`	Reset general.
`LED0`	Temporizador activo.
`LED1`	Alarma.
Displays 3 y 2	Tiempo.
Displays 1 y 0	Eventos.

La asignación exacta deberá realizarse mediante el archivo XDC correspondiente a la placa Nexys 4 DDR.

6. Arquitectura del sistema

El diseño deberá dividirse en módulos independientes.



7. Módulos requeridos

El proyecto deberá contener, como mínimo:

```
generador_ticks.vhd
detector_flanco.vhd
temporizador_bcd.vhd
contador_eventos.vhd
bcd7seg.vhd
mux_display.vhd
lab04_top.vhd
tb_lab04.vhd
lab04.xdc
```

8. Fundamentos principales

8.1. Generación de un segundo

La placa utiliza un reloj de:

$$f = 100\,\text{MHz}$$

Por lo tanto, para obtener una habilitación cada segundo se requieren:

$$N = 100\,000\,000$$

ciclos de reloj.

Se recomienda mantener el reloj principal y generar una señal `TICK_1S` activa durante un solo ciclo.

8.2. Contadores BCD

El tiempo y los eventos deberán representarse mediante dos dígitos BCD:

- unidades: 0 a 9;
- decenas: 0 a 9.

Ejemplo de conteo regresivo:

$$12 \rightarrow 11 \rightarrow 10 \rightarrow 09 \rightarrow 08$$

Cuando las unidades sean cero, se deberá disminuir la decena y cargar las unidades con nueve.

8.3. Multiplexación

Los displays comparten las líneas de segmentos, por lo que deben activarse uno a uno a alta velocidad.

Display	Información
0	Unidades de eventos.
1	Decenas de eventos.
2	Unidades de tiempo.
3	Decenas de tiempo.

Los displays restantes deberán permanecer apagados.

8.4. Tratamiento de pulsadores

Los pulsadores no deben utilizarse directamente como señales de reloj. Se deberá implementar:

- sincronización;
- detección de flanco;
- generación de un pulso de un ciclo.

Ejemplo conceptual:

```
pulso_evento <= evento_actual and not evento_anterior;
```

9. Trabajo previo

Antes del laboratorio, cada grupo deberá preparar:

1. diagrama de bloques;
2. definición de entradas y salidas;
3. algoritmo del temporizador regresivo;
4. algoritmo del contador de eventos;
5. tabla de visualización de los cuatro displays;
6. entidades VHDL propuestas;
7. casos de prueba;
8. explicación de contador BCD, generación de ticks, multiplexación, detección de flanco y rebote.

10. Desarrollo del laboratorio

10.1. Creación del proyecto

1. Inicie Vivado.
2. Cree un proyecto RTL.
3. Seleccione VHDL.
4. Seleccione el dispositivo `XC7A100TCSG324-1`.
5. Utilice como nombre `LAB04_TEMPORIZADOR_EVENTOS`.

10.2. Generador de ticks

El módulo deberá generar `TICK_1S` y `TICK_DISPLAY`.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity generador_ticks is
    Port (
        CLK          : in  STD_LOGIC;
        RESET        : in  STD_LOGIC;
        TICK_1S      : out STD_LOGIC;
        TICK_DISPLAY : out STD_LOGIC
    );
end generador_ticks;
```

Para la simulación deberán utilizarse valores reducidos, evitando simular 100 millones de ciclos.

10.3. Inicio y pausa

Cada pulsación en `START_PAUSE` deberá cambiar el estado:

$$\text{detenido} \rightarrow \text{ejecutándose} \rightarrow \text{detenido}$$

La señal `RUNNING` indicará si el temporizador está activo.

10.4. Temporizador regresivo

El temporizador deberá contar:

$$25 \to 24 \to 23 \to \cdots \to 01 \to 00$$

Cuando llegue a `00`:

- deberá detenerse;
- deberá activar `ALARMA`;
- no deberá continuar disminuyendo.

10.5. Contador de eventos

El contador deberá aumentar con cada pulsación válida:

$$00 \to 01 \to 02 \to \cdots \to 99$$

Al llegar a `99`, se recomienda mantener el valor.

10.6. Decodificador BCD a siete segmentos

Entidad sugerida:

```
entity bcd7seg is
    Port (
        BCD : in  STD_LOGIC_VECTOR(3 downto 0);
        SEG : out STD_LOGIC_VECTOR(6 downto 0)
    );
end bcd7seg;
```

Debe considerarse que los displays de la Nexys 4 DDR son activos en bajo.

10.7. Multiplexación

El módulo deberá seleccionar los cuatro dígitos de forma cíclica y enviar el valor correspondiente al decodificador.

La frecuencia de refresco deberá ser suficientemente alta para evitar parpadeo visible.

10.8. Integración

El módulo superior deberá conectar:

- generador de ticks;
- detectores de flanco;
- temporizador;
- contador de eventos;
- multiplexor;
- decodificador;
- LED, pulsadores y displays.

11. Banco de pruebas

El testbench deberá:

1. generar el reloj;
2. aplicar reset;
3. iniciar el temporizador;
4. verificar decrementos;
5. pausar;
6. reanudar;
7. incrementar eventos;
8. reiniciar eventos;
9. verificar la transición `10` a `09`;
10. verificar la llegada a `00`;
11. comprobar la alarma;
12. utilizar sentencias `assert`.

12. Casos mínimos de prueba

Caso	Acción	Resultado esperado
1	Reset general	Tiempo y eventos reiniciados.
2	Inicio	El temporizador comienza.
3	Pausa	El tiempo se mantiene.
4	Reanudación	El conteo continúa.
5	Evento	El contador aumenta una unidad.
6	Reset de evento	El contador vuelve a `00`.
7	Transición `10` a `09`	El cambio BCD es correcto.
8	Transición `01` a `00`	El temporizador finaliza.
9	Tiempo en `00`	La alarma se activa.
10	Evento en `99`	El contador mantiene `99`.
11	Reset de tiempo	Recupera el valor inicial.

13. Simulación

Durante la simulación deberán observarse:

- `CLK`;
- `RESET`;
- `TICK_1S`;
- `TICK_DISPLAY`;
- `START_PAUSE`;
- `RUNNING`;

- decenas y unidades de tiempo;
- decenas y unidades de eventos;
- `ALARMA`;
- `SEG`;
- `AN`.

El informe deberá incluir capturas de reset, inicio del conteo, pausa, reanudación, transición `10` a `09`, llegada a cero, alarma, incremento de eventos y multiplexación.

14. Síntesis e implementación

1. Ejecute *Run Synthesis*.
2. Revise errores y advertencias.
3. Abra el esquema RTL.
4. Identifique registros, contadores, comparadores y multiplexores.
5. Ejecute *Run Implementation*.
6. Genere el bitstream.
7. Programe la FPGA.
8. Verifique el sistema físicamente.

15. Verificación en hardware

El grupo deberá demostrar:

- funcionamiento del reset;
- inicio y pausa;
- decremento cada segundo;
- transición correcta entre dígitos;
- incremento de eventos;
- reinicio independiente;
- visualización de cuatro dígitos;
- detención en cero;
- activación de alarma;
- ausencia de conteos múltiples por pulsación.

16. Actividad adicional

Cada grupo deberá implementar una extensión:

- configuración del tiempo mediante interruptores;
- antirrebote de pulsadores;

- segundo contador de eventos;
- formato minutos y segundos;
- alarma intermitente;
- control del sistema mediante una FSM.

17. Preguntas de análisis

1. ¿Por qué se utiliza una señal de habilitación y no un nuevo reloj?
2. ¿Cuántos ciclos de 100 MHz equivalen a un segundo?
3. ¿Por qué deben reducirse los valores del divisor durante la simulación?
4. ¿Qué diferencia existe entre un contador binario y uno BCD?
5. ¿Cómo se realiza la transición `10` a `09`?
6. ¿Por qué el temporizador debe detenerse en cero?
7. ¿Qué función cumple `RUNNING`?
8. ¿Por qué se requiere detección de flanco?
9. ¿Qué efecto produce el rebote de un pulsador?
10. ¿Cómo funciona la multiplexación?
11. ¿Qué ocurre si la frecuencia de refresco es demasiado baja?
12. ¿Cómo podría ampliarse el temporizador a minutos y segundos?

18. Contenido del informe

El informe deberá incluir:

1. portada;
2. introducción;
3. objetivos;
4. fundamentos teóricos;
5. descripción funcional;
6. diagrama de bloques;
7. código VHDL;
8. testbench;
9. capturas de simulación;
10. archivo XDC;
11. esquema RTL;
12. resultados en hardware;
13. análisis de errores;
14. actividad adicional;

15. respuestas a preguntas;
16. conclusiones;
17. referencias.

19. Evidencias obligatorias

Cada grupo deberá entregar:

- informe en PDF;
- archivos VHDL;
- testbench;
- archivo XDC;
- diagrama de bloques;
- capturas de simulación;
- reporte de síntesis;
- fotografía o video de funcionamiento;
- proyecto comprimido de Vivado.

20. Criterios de evaluación

Criterio	Ponderación
Comprensión y fundamentación teórica	20 %
Diseño, metodología y ejecución experimental	25 %
Análisis e interpretación de resultados	25 %
Comparación de resultados con la teoría	15 %
Conclusiones y presentación del informe	15 %
Total	**100 %**

21. Lista de comprobación

☐ El temporizador inicia correctamente.

☐ Puede pausarse y reanudarse.

☐ El conteo disminuye cada segundo.

☐ La transición `10` a `09` funciona.

☐ El temporizador se detiene en `00`.

☐ La alarma se activa.

☐ El contador aumenta una vez por pulsación.

☐ El contador puede reiniciarse.

☐ Los cuatro dígitos se muestran correctamente.

☐ No existe parpadeo visible.

- ☐ El testbench utiliza `assert`.
- ☐ La síntesis no presenta errores.
- ☐ El archivo XDC coincide con los puertos.
- ☐ El sistema funciona en la FPGA.
- ☐ El informe contiene todas las evidencias.